

Assignment - 03

Title : Circular Queue.

Objective : To study data Structure and their implementation using Circular Queue.

Problem Statement :

Implement Circular Queue Using array. perform following operation on it.

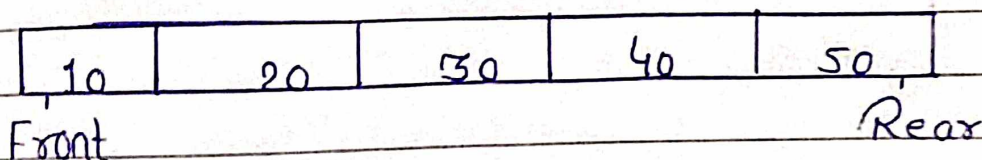
- Insertion (Enqueue)
- Deletion (Dequeue)
- Display

(Note :- Handle queue full condition by considering a fixed size of a queue.)

Out Come : Design algorithm based on techniques like :- Circular Queue.

Theory :

- Queue's $\Rightarrow$ The queue can be formally defined as ordered collection of elements that has two ends named as Front and rear. From the front end one can delete the elements and from the rear end one can insert the elements.



Operation's :

1. Insert : The insertion of the element in the queue is done by the end called rear. Before the insertion of the element it is checked whether or not the queue is full.

2. Delete : The deletion of the element from the queue is done by the end called front. Before performing the delete operation it is checked whether the queue is empty or not.

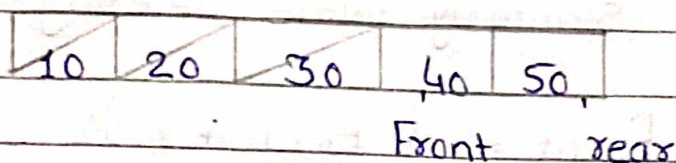
The Queue is also called as FIFO i.e. First In First Out data structure.

3. Queue Overflow : (Queue full condition) If the rear pointer is going beyond the Maximum size of the queue then the queue overflow occurs.

4. Queue Underflow : (Queue empty condition) If the queue is empty, we cannot perform the deletion. The result of attempt to delete an element from the empty queue is called the queue underflow condition.

• Concept of Circular Queue :

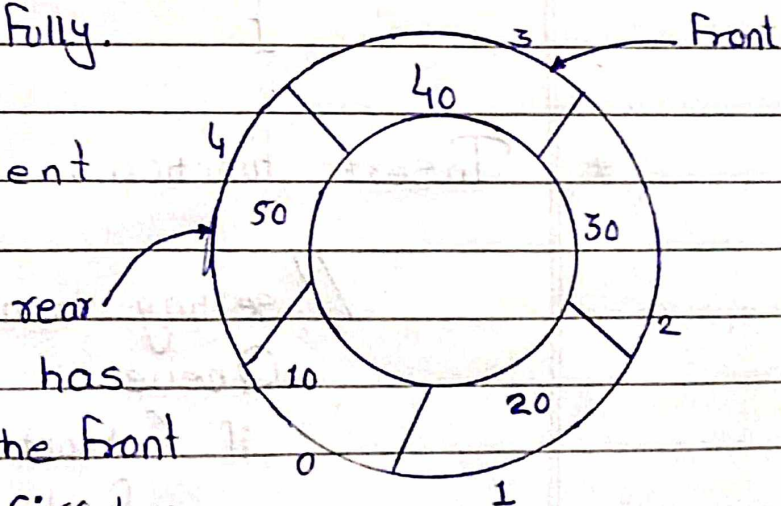
In Case of linear queue the element's get deleted logically.



In the above given linear queue we have deleted the element 10, 20, 30. Mean's Simply the Front pointer is Shifted ahead. We always Consider the queue from Front to rear. and if ~~the~~ We try to insert any other element in the queue then It will give us the "queue full" Message. also there is a Space of element 10, 20 and 30 (these are deleted element), we can not utilize them because queue is nothing but a linear array.

Hence there is a Concept Called Circular queue. The main advantage of Circular queue is that we can Utilize the space of the queue Fully.

Considering that the element deleted are 10, 20 & 30.



There is a formula which has to be applied for setting the Front and rear pointers, for a Circular queue.

$$\text{rear} = (\text{rear} + 1) \% \text{Size}$$

$$\text{Front} = (\text{Front} + 1) \% \text{Size}$$

$$\text{rear} = (\text{rear} + 1) \% \text{Size}$$

$$= (4 + 1) \% 5$$

$$\text{rear} = 0$$

So, we can store the element 60 at the 0th location similarly while deleting the element

$$\text{front} = (\text{front} + 1) \% \text{Size}$$

$$= (3 + 1) \% 5$$

$$\text{front} = 4$$

So, delete the element at 4th location i.e. element 50.

* Queue full Condition :-

```

if (front == (rear + 1) % Size)
{
    cout << "Queue is full\n";
}

```

* Insert function :-

// setting front pointer for a single element in Queue.

```

if (front == -1)

```

```

    front = rear = 0;

```

```

else

```

```

    rear = (rear + 1) % Size;

```

```

    Que[rear] = item;

```

* Queue empty Condition :-

```
{ if (Front == -1)
    cout << " Queue is empty" ;
    return 0 ;
}
```

* Delet Function :-

```
val = Que [Front] ; // item to be deleted.
{ if (Front == rear) // When single element is present
    Front = rear = -1 ;
}
else
{ Front = (Front + 1) % Size ;
  return val ;
}
```

Test Case	Test Case description	Test data	Expected Result	Actual Result	pass/fail
1.	Check the result When user Select option to insert an element in Circular queue.	Add the element into the Circular Queue	New element get inserted into the Circular queue	New element get inserted in the Circular queue	pass
2.	check the result When user Select the option to Delete the element from Circular queue.	delete's element from Circular queue	delete the element	element get deleted from the Circular queue	pass
3.	check the result When user Select the option of display the Circular queue.	Display element	All the element of Circular queue get displayed	All the element's of queue (Circular Q) get display	pass

```
#include<iostream>
```

```
#define size 10
```

```
using namespace std;
```

```
class Queue
```

```
{
```

```
int Que[size];
```

```
int front, rear;
```

```
public:
```

```
Queue()//constructor defined
```

```
{
```

```
front=-1;
```

```
rear=0;
```

```
}
```

```
void init();
```

```
void insert(int ch);
```

```
int delet();
```

```
void display();
```

```
};
```

```
void Queue::init()
```

```
{
```

```
int i;
```

```
for(i=0;i< size;i++)
```

```

    Que[i]=0;
}

void Queue::insert(int item)
{
    if (front==(rear+1)% size)
    {
        cout<<"Queue is full\n";
    }
    else
    {
        //setting front pointer for a single element in Queue
        if(front==-1)
            front=rear=0;
        else
            rear=(rear+1)% size; Que[rear]=item;
    }
}

int Queue::delet()
{
    int val;
    if(front==-1)
    {
        cout << "Queue is empty\n";
        return 0; // return null on empty Queue
    }
}

```

```
val= Que[front]; //item to be deleted
if(front==rear) //when single element is present
{
front=rear=-1;
}
else
front=(front+1)%size;
return val;
}
```

```
void Queue::display()
{
int i;
i=front;
while(i!=rear)
{
cout<<Que[i]<<" ";
i=(i+1)%size;
}
cout<<Que[i]<<endl;
}
```

```
int main()
{
int i,choice,item;
char ans ='y';
Queue obj;
```

```
obj.init();  
do  
{  
    cout<<" Menu"<<endl<<endl;  
    cout<<"1.Insert \n 2.Delete \n 3.Display"<<endl;  
    cout<<"Enter Your choice"<<endl<<endl;  
    cin>>choice;  
  
    switch(choice)  
    {  
        case 1:cout<<"Enter the element"<<endl;  
        cin>>item;  
        obj.insert(item);  
        break;  
        case 2:cout<<"The element to be deleted is"<<endl;  
        cout<<obj.delet();  
        break;  
        case 3:obj.display();  
        break;  
    }  
  
}while (choice!=0);  
return 0;  
}
```